Week 1 Documentation CCC

Name: Rohit Chintalapudi
Squad: Squad 15
Team name: AI Avengers
Duration: 29th May - 4th June

Day 1 - 29th May

My Overview & Learning POV

Today marked the official launch of our CCC internship program. As the Squad Lead, my primary focus was helping our mentor, Pujitha Mam, map out our squad's overall skills, backgrounds, and coding experience. Mam explained the high-level roadmap, the technical frameworks we will use, and how the program is structured. The biggest takeaway was that our final project must be a highly collaborative group effort. Clear communication and joint responsibility are crucial for our success.

Key Discussion Points

- **Team Mapping:** We shared our backgrounds, programming skill sets, and academic focus areas to help plan out our tasks.
- **Program Roadmap:** Mam outlined the project expectations, timeline milestones, and the software evaluation criteria.
- **Team Alignment:** We learned how our individual progress directly affects our overall group delivery.

Action Items

- Open up internal communication channels for Squad 15 to make collaboration smoother.
- Review the initial project parameters and foundational roadmaps shared by Mam.

Day 2 - 30th May

My Overview & Learning POV

Today, we shifted our focus to setting up our development standards. Mam emphasized that mastering version control is absolutely non-negotiable if we want to prevent code loss and repository mess-ups when we all start coding together. Our team was given an exhaustive assignment to deep-dive into Git workflows. I spent time practicing core Git tracking steps and making sure everyone on the team could log into our tracking systems.

Key Discussion Points

- **Git Standards:** We looked at the absolute need for proper version control before touching a shared codebase.
- **Accountability:** Mam set up strict rules for our day-to-day progress logging and milestone tracking.

Formulas & Technical Logic

Essential Git Workflow Commands:

```shell
# 1. Initialize repository locally
git init

# 2. Stage changes (add files to the staging area)
git add .

# 3. Commit staged changes (create a snapshot in the local repository)
git commit -m "feat: Add initial project structure"

# 4. Pull changes from remote (fetch and merge)
git pull origin main

# 5. Push local commits to the remote repository
git push origin <branch-name>
```

Action Items

- Run local command tests to practice branch management and resolve merge conflicts.

Day 3 - 1st June

My Overview & Learning POV

After the weekend, we got together to solve the active Git and branching issues our team ran into. Mam walked us through exact command behaviors and explained repository hygiene, showing us how clean branching and well-structured commit histories stop code from getting overwritten. Once our Git doubts were cleared, Mam officially introduced **TypeScript** into our development stack. We discussed why its strict type safety is essential for building large, scalable projects.

Key Discussion Points

- **Git Troubleshooting:** Solved version control blocks and clarified staging errors.
- **Why TypeScript?:** Learned how static typing helps catch bugs during development rather than at runtime.
- **Roadmap Setup:** Received our next learning goals, which include setting up our local TypeScript environments, understanding control flows, and working with typed arrays.

Formulas & Technical Logic

TypeScript Basic Type Annotation:

```typescript
// Explicitly define a variable's type (Type Safety)
let squadId: number = 15;
let teamName: string = "AI Avengers";

// Function with input and output type annotations
function greetMember(name: string): string {
  return `Hello, ${name} of Squad ${squadId}!`;
}
```

Action Items

- Apply the new Git best practices and branch rules directly to our active work environments.
- Install, set up, and verify a working TypeScript environment locally.
- Practice basic TypeScript logic, loops, and typed array structure

---

Day 4 - 2st June

My Overview & Learning POV

Today was a major checkpoint. Mam conducted a detailed technical evaluation to check how well we understood foundational TypeScript. Every team member was checked on environment setup, primitive types (any, unknown), operators, control flows, and function definitions. While we proved we understood the basics, the review showed that a few of us need a bit more practice to close some alignment gaps. Right after, Mam pushed us into advanced data structuring, telling us to immediately start mastering objects, interfaces, and enums.

Key Discussion Points

- **Syntax Evaluation:** Checked how the team handles variables, conditional statements, loops, and arrow functions.

- **Advanced Onboarding:** Shifted our focus to complex schemas, interfaces, type aliases, and tuples to get ready for upcoming code integration.

Formulas & Technical Logic

TypeScript Interfaces, Type Aliases, and Tuples:

```typescript
// Interface for defining the structure of an object
interface Member {
  id: number;
  name: string;
  role: "Frontend" | "Backend";
}

// Type Alias for defining a custom type name
type Coordinates = {
  x: number;
  y: number;
};

// Tuple structure (fixed length array with ordered, specific types)
let memberTuple: [string, number] = ["Rohit", 15];

// Using the Interface
const myDetails: Member = {
  id: 15,
  name: "Rohit Chintalapudi",
  role: "Backend"
};
```

Action Items

- Team members who struggled with the basics must review syntax, types, and functions immediately.
- Deep-dive into Objects, Interfaces, Type Aliases, Enums, Arrays, and Tuples.
- Prepare local code examples to show proficiency in these data structures during the next review.

Day 5 - 3st June

My Overview & Learning POV

We started today by reviewing the advanced TypeScript concepts assigned yesterday, checking our understanding of interfaces, aliases, enums, and tuples. Then came the big moment: Mam announced the name of our official project—**AI Doubt Clarification**. We brainstormed different problem statements and split our squad into Frontend and Backend sub-teams to organize the work. To wrap up, Mam introduced us to running AI models locally using Ollama and automation tools like n8n.

Key Discussion Points

- **TypeScript Check:** Reviewed data collections and complex types.
- **Project Kickoff:** Introduced the *AI Doubt Clarification* project and brainstormed features.
- **Team Division:** Divided our squad into frontend and backend groups for better focus.
- **Local AI Infrastructure:** Introduced local model execution tools to get us off cloud dependencies.

Formulas & Technical Logic

Local AI Architecture Concept (Request Flow):

```
None
[User Request]
  |
  V
[Frontend UI]
  |
  V
[Backend (Node.js)]
  |
  V
[Ollama (Local LLM Execution)]
  |
  V
[Local Model Response]
```

Action Items

- Finalize the exact frontend and backend team splits.
- Download and install **Ollama** on our local machines.
- Research and understand **MCP (Model Context Protocol)**, **Agent SDKs**, and **n8n workflow automation**.

Day 6 - 4th June

My Overview & Learning POV

We wrapped up our first week by narrowing down our project direction. The team shared the problem statements we thought of, and Mam helped us pick out the best core idea for our *AI Doubt Clarification* app. On the technical side, I worked with the team to check that everyone had Ollama installed correctly. We focused on testing our local Ollama setups with **Node.js** to make sure our backend can speak to the AI model smoothly. Mam also encouraged us to explore other open-source models available in the Ollama library to see which ones perform best.

Key Discussion Points

- **Problem Statement Selection:** Refined and picked our final project problem statement.
- **Environment Verification:** Checked local installations of Ollama and verified Node.js environment hooks.
- **Model Exploration:** Discussed trying out different open-source models to find the right balance between speed and accuracy.

Formulas & Technical Logic

Asynchronous Backend Connection Logic:

JavaScript

```
// Example Node.js function using an Ollama client library
async function generateResponse(prompt) {
  try {
    // Await the asynchronous model generation call
    const response = await ollama.generate({
      model: 'llama2', // Model pulled from Ollama library
      prompt: prompt,
    });

    // Process and return the JSON response stream
    return response.json();

  } catch (error) {
    console.error("Ollama connection failed:", error);
    return null;
  }
}
```

Action Items

- Anyone who hasn't finished connecting Node.js with Ollama must finish and test their setup right away.
- Browse the Ollama library, pull different models, and test how they run on local hardware.